

PORTADA

Técnicas de Programación

Unidad 2 — Paradigma Objetual: Interfaces y Polimorfismo (Parte 2)

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

El downcasting que viste la sesión pasada puede reventar tu programa en pleno tiempo de ejecución.



TRANSICIÓN

De "casi siempre funciona" a "siempre funciona"

La sesión pasada viste que `(Avion) v` lanza `ClassCastException` si el objeto real no es un `Avion`. El problema es que, al recorrer una colección mixta, no siempre sabes con certeza qué tipo concreto trae cada elemento antes de convertirlo. Hoy resolvemos eso, y llevamos el polimorfismo a herramientas que usarás todo el semestre: `ArrayList` y la interfaz `Comparable` de Java.

CONCEPTO

El operador `instanceof` : preguntar antes de convertir

El operador `instanceof` pregunta en tiempo de ejecución si un objeto es (o hereda de, o implementa) un tipo determinado, devolviendo `true` / `false` . Con esto, el downcasting deja de ser una apuesta.

```
Volador v = new Avion();

if (v instanceof Avion) {
    Avion a = (Avion) v;
    a.aterrizarEnPista();    // ahora es seguro: ya confirmamos el tipo
}
```

CONCEPTO

Pattern matching: la forma corta de instanceof

Desde versiones recientes de Java, `instanceof` puede declarar la variable convertida en la misma línea de la comprobación, evitando el `(Avion) v` repetido.

```
if (v instanceof Avion a) {  
    a.atterrizarResultado(); // "a" ya viene convertido a Avion, listo para usar  
}
```

Ambas formas son válidas — usa la que tu IDE/versión de Java soporte mejor.

CONTRA EJEMPLO

Lo que `instanceof` no reemplaza

Encadenar muchos `if (x instanceof Tipo)` para decidir qué hacer con cada elemento es una señal de que el diseño está apoyándose en la jerarquía de tipos en vez de en el contrato. Si te encuentras haciendo esto en cada método, probablemente falta un método en la interfaz que resuelva ese comportamiento de forma polimórfica, sin preguntar el tipo.

```
// Evitar esto cuando se pueda:  
if (v instanceof Ave) { /* ... */ }  
else if (v instanceof Avion) { /* ... */ }  
// Mejor: que volar() ya resuelva el comportamiento distinto por sí mismo
```

CONCEPTO

Polimorfismo con ArrayList , no solo con arreglos

Todo lo que aprendiste con arreglos de interfaces aplica igual — y mejor — con `ArrayList` , que es la colección que más vas a usar de aquí en adelante. Declarar la lista con el tipo de la interfaz permite guardar objetos de cualquier clase que la implemente.

```
ArrayList<Volador> voladores = new ArrayList<>();
voladores.add(new Ave());
voladores.add(new Avion());

for (Volador v : voladores) {
    v.volar();    // cada uno resuelve su propia versión, igual que con arreglos
}
```

CASO

Un ejemplo real del JDK: `Comparable<T>`

Java trae, desde su librería estándar, una interfaz llamada `Comparable<T>` con un único método: `compareTo(T otro)` .
Cualquier clase que la implemente le dice a Java **cómo ordenar** sus objetos — y por eso métodos como `Collections.sort()` funcionan con cualquier tipo, sin que Java necesite conocerlo de antemano.

CONCEPTO

Implementando Comparable<T>

`compareTo(otro)` debe devolver un número **negativo** si el objeto actual va antes, **positivo** si va después, y **cero** si son equivalentes en el orden.

```
public class Estudiante implements Comparable<Estudiante> {  
    private double promedio;  
  
    @Override  
    public int compareTo(Estudiante otro) {  
        return Double.compare(this.promedio, otro.promedio);  
    }  
}
```

Con esto ya implementado, `Collections.sort(listaDeEstudiantes)` ordena la lista completa de menor a mayor promedio automáticamente.

TENSIÓN CONCEPTUAL

¿Muchas interfaces pequeñas, o una sola grande?

Una interfaz con muchos métodos obliga a cada clase que la implemente a definirlos **todos**, aunque algunos no le apliquen — eso rompe el propósito del contrato. Es preferible dividir el comportamiento en interfaces pequeñas y específicas (como `Volador` , `Nadador` , `Comparable`) e implementar solo las que la clase realmente necesita, en vez de una interfaz enorme tipo "hace de todo".

INTERACCIÓN

Ordenen su propia lista polimórfica

En parejas: retomen la interfaz `Pagable` con `Factura` y `Nomina` de la sesión pasada. Agréguele `Comparable<Pagable>` a ambas clases, comparando por `calcularMonto()`, y describan qué imprimiría `Collections.sort()` sobre un `ArrayList<Pagable>` con ambos tipos mezclados.

6 minutos · estén listos para explicar qué devuelve `compareTo()` en cada comparación

SÍNTESIS

Lo que hay que llevarse de hoy

1. `instanceof` permite verificar el tipo real de un objeto antes de convertirlo, evitando `ClassCastException` .
2. Encadenar muchos `instanceof` es una señal de que falta resolver el comportamiento de forma polimórfica dentro de la interfaz.
3. El polimorfismo con `ArrayList<Interfaz>` funciona igual que con arreglos, y es la forma que más usarás en el curso.
4. `Comparable<T>` es un ejemplo real de interfaz del JDK: implementarla le dice a Java cómo ordenar tus objetos con `Collections.sort()` .



CIERRE

Antes de la próxima sesión

- Implementar `Comparable<Estudiante>` (o un ejemplo propio) y probar `Collections.sort()` sobre una lista con varios objetos.
- Reescribir un `if (x instanceof Tipo)` encadenado propio como método polimórfico dentro de una interfaz.

La próxima sesión cerramos el bloque de interacción con el usuario: entrada y salida de información en Java.

